

[← Back to calendar](#)

0% progress

 **Mandatory deliverable**

LAB | Configure Remote State Lab

Skills you will learn

Infrastructure as Code: Terraform

AWS: S3

AWS: DynamoDB

Lab M4.03 - Remote State & Backends

Repository:<https://github.com/cloud-engineering-bootcamp/ce-lab-terraform-remote-state>**Activity Type:** Individual **Estimated Time:** 45-60 minutes **Submission:** GitHub Repository

Learning Objectives

- ☐ Configure S3 backend for Terraform state
- ☐ Set up DynamoDB table for state locking
- ☐ Migrate local state to remote backend
- ☐ Understand state locking and concurrency
- ☐ Implement best practices for state management
- ☐ Secure state files with encryption

Prerequisites

- ☐ Completed Lab M4.02 (Variables)
- ☐ AWS account with S3 and DynamoDB permissions
- ☐ Understanding of Terraform state concepts

Introduction

Local state files work for individual developers, but teams need shared state. What happens when two people run `terraform apply` simultaneously? Disaster! This lab teaches you how to use remote state with locking to prevent conflicts.



Scenario

Your team is growing. Multiple engineers need to work on the same infrastructure. You need:

- **Shared state** - Everyone sees the same infrastructure state
- **State locking** - Prevent concurrent modifications
- **Encryption** - Protect sensitive data in state
- **Versioning** - Track state changes over time

Your Task

What you'll create:

- S3 bucket for Terraform state
- DynamoDB table for state locking
- Backend configuration
- Encrypted, versioned remote state
- Demonstration of concurrent access protection

Success criteria:

- ☐ S3 bucket created with encryption and versioning
- ☐ DynamoDB table created for locking
- ☐ State migrated from local to S3
- ☐ Lock mechanism tested and working
- ☐ Documentation complete

Time limit: 45-60 minutes

Step-by-Step Instructions

Step 1: Create Backend Infrastructure

Create `backend-setup/main.tf`:

```
terraform {  
  required_version = ">= 1.6.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```



```
provider "aws" {
  region = "us-east-1"
}

# S3 bucket for state
resource "aws_s3_bucket" "terraform_state" {
  bucket = "your-name-terraform-state-bucket" # Change this!

  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Name      = "Terraform State Bucket"
    Environment = "Shared"
    ManagedBy  = "Terraform"
  }
}

# Enable versioning
resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Enable encryption
resource "aws_s3_bucket_server_side_encryption_configuration"
"terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# Block public access
resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# DynamoDB table for locking
```



```
resource "aws_dynamodb_table" "terraform_locks" {
  name      = "terraform-state-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key  = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name      = "Terraform State Locks"
    Environment = "Shared"
    ManagedBy  = "Terraform"
  }
}

# Outputs
output "s3_bucket_name" {
  value = aws_s3_bucket.terraform_state.id
}

output "dynamodb_table_name" {
  value = aws_dynamodb_table.terraform_locks.name
}
```

Deploy backend infrastructure:

```
cd backend-setup
terraform init
terraform plan
terraform apply
```

[Copy](#)[Explain](#)

Expected outcome: S3 bucket and DynamoDB table created.

Step 2: Create Main Project with Local State

Create main-project/main.tf:

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```



```
provider "aws" {  
  region = var.aws_region  
}  
  
resource "aws_s3_bucket" "example" {  
  bucket = "${var.project_name}-example-bucket"  
  
  tags = {  
    Name      = "${var.project_name}-bucket"  
    Environment = var.environment  
  }  
}
```

Create main-project/variables.tf:

```
variable "aws_region" {  
  default = "us-east-1"  
}  
  
variable "project_name" {  
  default = "myproject"  
}  
  
variable "environment" {  
  default = "dev"  
}
```

 Copy Explain

Deploy with local state:

```
cd main-project  
terraform init  
terraform apply
```

 Copy Explain

Check state location:

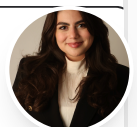
```
ls -la terraform.tfstate  
# State is stored locally
```

 Copy Explain

Step 3: Configure Remote Backend

Add backend configuration to main-project/main.tf:

```
terraform {  
  required_version = ">= 1.6.0"
```



```
backend "s3" {  
  bucket      = "your-name-terraform-state-bucket" # Change this!  
  key         = "main-project/terraform.tfstate"  
  region      = "us-east-1"  
  encrypt     = true  
  dynamodb_table = "terraform-state-locks"  
}  
  
required_providers {  
  aws = {  
    source = "hashicorp/aws"  
    version = "~> 5.0"  
  }  
}
```

Step 4: Migrate State to Remote Backend

Re-initialize with new backend
`terraform init -migrate-state`



Copy



Explain

Terraform will ask: "Do you want to copy existing state to the new backend?"
Type: yes

Expected output:

```
Initializing the backend...  
Do you want to copy existing state to the new backend?  
Pre-existing state was found while migrating the previous "local"  
newly configured "s3" backend. No existing state was found in the  
configured "s3" backend. Do you want to copy this state to the ne  
backend? Enter "yes" to copy and "no" to start with an empty stat
```

Enter a value: yes

Successfully configured the backend "s3"!

Verify migration:

Check S3 bucket
`aws s3 ls s3://your-name-terraform-state-bucket/main-project/`



```
# Should show: terraform.tfstate
```

```
# Local state should be backed up  
ls terraform.tfstate.backup
```

Step 5: Test State Locking

Create `test-locking.sh`:

```
#!/bin/bash
```

[Copy](#)[Explain](#)

```
echo "Testing state locking..."  
echo ""  
echo "Step 1: Start terraform apply in background (don't auto-  
approve)"  
terraform apply &  
FIRST_PID=$!  
  
sleep 3  
  
echo ""  
echo "Step 2: Try to run another terraform apply"  
echo "This should fail with a lock error"  
terraform plan  
  
echo ""  
echo "Step 3: Kill first process"  
kill $FIRST_PID  
  
echo ""  
echo "Step 4: Verify lock is released"  
terraform plan
```

Run test:

```
chmod +x test-locking.sh  
./test-locking.sh
```

[Copy](#)[Explain](#)

Expected behavior: Second command fails with lock error.

Step 6: Check DynamoDB Lock Table

```
# View lock table items (when locked)  
aws dynamodb scan --table-name terraform-state-locks  
  
# Example output when locked:
```



```
# {  
# "LockID": "your-name-terraform-state-bucket/main-  
project/terraform.tfstate",  
# "Info": "..."  
# }
```

Step 7: State Versioning Demonstration

Make a change

```
echo 'resource "aws_s3_bucket" "another" {  
  bucket = "another-example-bucket"  
}' >> main.tf
```

[Copy](#)[Explain](#)

Apply change

```
terraform apply -auto-approve
```

Check S3 versions

```
aws s3api list-object-versions \  
  --bucket your-name-terraform-state-bucket \  
  --prefix main-project/terraform.tfstate
```

Should show multiple versions

Step 8: Recover from Version History

List versions with timestamps

```
aws s3api list-object-versions \  
  --bucket your-name-terraform-state-bucket \  
  --prefix main-project/terraform.tfstate \  
  --query 'Versions[*].[VersionId,LastModified]' \  
  --output table
```

[Copy](#)[Explain](#)

Download specific version

```
aws s3api get-object \  
  --bucket your-name-terraform-state-bucket \  
  --key main-project/terraform.tfstate \  
  --version-id <VERSION_ID> \  
  terraform.tfstate.old
```

Step 9: Best Practices Implementation

Create .gitignore:

```
# Terraform  
*.tfstate  
*.tfstate.*
```




```
.terraform/  
.terraform.lock.hcl  
*.tfvars  
!terraform.tfvars.example
```

Create terraform.tfvars.example :

```
# Example configuration  
# Copy to terraform.tfvars and customize
```

[Copy](#)[Explain](#)

```
aws_region = "us-east-1"  
project_name = "myproject"  
environment = "dev"
```

Troubleshooting

Issue 1: Lock Timeout

Error:

```
Error: Error acquiring the state lock  
Lock Info:  
  ID:          abc-123  
  Path:        ...  
  Operation:   OperationTypeApply  
  Who:         user@hostname
```

Solution:

```
# Force unlock (use carefully!)  
terraform force-unlock abc-123
```

[Copy](#)[Explain](#)

Issue 2: Backend Initialization Failed

Error:

```
Error: Failed to get existing workspaces: S3 bucket does not exist.
```

Solution: Ensure S3 bucket exists and bucket name in backend config is correct.



Issue 3: Access Denied

Error:

Error: error listing S3 Bucket Objects: Access Denied

Solution: Check AWS credentials and IAM permissions for S3 and DynamoDB.

Documentation & Submission

Create README.md

Lab M4.03 - Remote State & Backends

[Copy](#)[Explain](#)

Overview

Configured remote state storage in S3 with DynamoDB locking.

Infrastructure

- **State Bucket:** your-name-terraform-state-bucket
- **Lock Table:** terraform-state-locks
- **Encryption:** AES256
- **Versioning:** Enabled

Backend Configuration

```
\\\`hcl
backend "s3" {
  bucket      = "your-name-terraform-state-bucket"
  key         = "main-project/terraform.tfstate"
  region      = "us-east-1"
  encrypt     = true
  dynamodb_table = "terraform-state-locks"
}
\\\`
```

Testing

State locking tested and verified working.
Version history enabled for state recovery.

Security

- Bucket encrypted at rest
- Public access blocked
- Versioning enabled
- State locking prevents concurrent modifications

Create Repository



```
git init
git add .
git commit -m "Lab M4.03: Remote State & Backends"
```

 Copy Explain

- Created S3 bucket for state storage
- Configured DynamoDB table for locking
- Migrated local state to remote backend
- Tested state locking mechanism
- Implemented versioning for state recovery"

```
gh repo create ce-lab-terraform-remote-state --public
git push -u origin main
```

Bonus Challenges

Challenge 1: Multiple Environments

Configure separate state files for dev/prod:

```
dev/terraform.tfstate
prod/terraform.tfstate
```

Challenge 2: KMS Encryption

Use AWS KMS instead of SSE-S3:

```
kms_key_id = "arn:aws:kms:us-east-1:123456789012:key/..."
```

 Copy Explain

Challenge 3: Cross-Account Backend

Set up backend in different AWS account.

Grading Rubric (100 points)

Criteria	Points
S3 bucket created with encryption	20
DynamoDB table configured	15
State migrated successfully	20
Locking tested and working	20



Criteria	Points
Documentation complete	15
Best practices implemented	10

Key Takeaways

✔ **Remote state** enables team collaboration ✔ **State locking** prevents conflicts ✔ **Encryption** protects sensitive data ✔ **Versioning** enables state recovery ✔ **S3 + DynamoDB** is the standard backend ✔ **Never commit** state files to git

Next Lab: M4.04 - State Management & Troubleshooting

LAB | Configure Remote State

Lab

URL

Enter the URL here

Submit lab

Previous

Next

